

Experiment ‘p_14_2_selection_sort_interval_array’ Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_14_2_selection_sort_interval_array

Natural language statement: null

Method signature: p_14_2_selection_sort_interval_array(inputs: array<int>, bound: nat) returns (sorted: array<int>)

Ensures

- forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]
- multiset(inputs[..]) == multiset(sorted[..])

Requires

- forall i :: 0 <= i < inputs.Length ==> 0 <= inputs[i] <= bound

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_14_2_selection_sort_interval_array(inputs: array<int>, bound: nat) returns (sorted:
↪ array<int>)

The method should respect the following contract:

requires forall i :: 0 <= i < inputs.Length ==> 0 <= inputs[i] <= bound, ensures forall i
↪ , j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j], ensures multiset(
↪ inputs[..]) == multiset(sorted[..])

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnPlqZF3KGimF7UX4jZnMnUpMZvjb

Model: gpt-5.5-2026-04-23

Created at: 1780669506

Finish reason: stop

Usage: 358 tokens in, and 10107 tokens out

```
{
  var n := inputs.Length;
  sorted := new int[n];

  var k := 0;
  while k < n
    invariant 0 <= k <= n
    invariant n == inputs.Length
    invariant sorted.Length == n
    invariant forall t :: 0 <= t < k ==> sorted[t] == inputs[t]
  {
    sorted[k] := inputs[k];
    k := k + 1;
  }
  assert forall t :: 0 <= t < n ==> sorted[t] == inputs[t];
  assert sorted[..] == inputs[..];

  var i := 0;
```

```

while i < n
  invariant 0 <= i <= n
  invariant n == inputs.Length
  invariant sorted.Length == n
  invariant multiset(sorted[..]) == multiset(inputs[..])
  invariant forall p, q :: 0 <= p < q < i ==> sorted[p] <= sorted[q]
  invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
{
  var min := i;
  var j := i + 1;
  while j < n
    invariant i + 1 <= j <= n
    invariant i <= min < j
    invariant sorted.Length == n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall p, q :: 0 <= p < q < i ==> sorted[p] <= sorted[q]
    invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
    invariant forall t :: i <= t < j ==> sorted[min] <= sorted[t]
  {
    if sorted[j] < sorted[min] {
      var oldMin := min;
      assert sorted[j] < sorted[oldMin];
      min := j;
      assert forall t :: i <= t < j ==> sorted[min] <= sorted[t] by {
        forall t | i <= t < j
          ensures sorted[min] <= sorted[t]
        {
          assert sorted[min] < sorted[oldMin];
          assert sorted[oldMin] <= sorted[t];
        }
      }
      assert sorted[min] <= sorted[j];
    } else {
      assert sorted[min] <= sorted[j];
    }
    assert forall t :: i <= t < j + 1 ==> sorted[min] <= sorted[t] by {
      forall t | i <= t < j + 1
        ensures sorted[min] <= sorted[t]
      {
        if t == j {
          assert sorted[min] <= sorted[j];
        } else {
          assert i <= t < j;
          assert sorted[min] <= sorted[t];
        }
      }
    }
    j := j + 1;
  }

  ghost var before := sorted[..];

```

```

assert |before| == n;
assert forall p, q :: 0 <= p < q < i ==> before[p] <= before[q];
assert forall p, q :: 0 <= p < i <= q < n ==> before[p] <= before[q];
assert forall t :: i <= t < n ==> before[min] <= before[t];

if min != i {
  assert i < min < n;
  sorted[i], sorted[min] := sorted[min], sorted[i];
  assert sorted[i] == before[min];
  assert sorted[min] == before[i];
  assert forall t :: 0 <= t < n && t != i && t != min ==> sorted[t] == before[t];
  assert sorted[..] == before[i := before[min]][min := before[i]];
  assert multiset(before[i := before[min]]) == multiset(before) - multiset{before[i]}
    ↪ + multiset{before[min]};
  assert multiset(before[i := before[min]][min := before[i]]) == multiset(before[i :=
    ↪ before[min]]) - multiset{before[min]} + multiset{before[i]};
  assert multiset(sorted[..]) == multiset(before);
} else {
  assert sorted[..] == before;
  assert sorted[i] == before[min];
  assert sorted[min] == before[i];
  assert forall t :: 0 <= t < n && t != i && t != min ==> sorted[t] == before[t];
  assert multiset(sorted[..]) == multiset(before);
}
assert multiset(sorted[..]) == multiset(inputs[..]);

assert forall p, q :: 0 <= p < q < i + 1 ==> sorted[p] <= sorted[q] by {
  forall p, q | 0 <= p < q < i + 1
    ensures sorted[p] <= sorted[q]
  {
    if q < i {
      assert p != i && p != min;
      assert q != i && q != min;
      assert sorted[p] == before[p];
      assert sorted[q] == before[q];
      assert before[p] <= before[q];
    } else {
      assert q == i;
      assert p < i;
      assert p != i && p != min;
      assert sorted[p] == before[p];
      assert sorted[q] == before[min];
      assert before[p] <= before[min];
    }
  }
}

assert forall p, q :: 0 <= p < i + 1 <= q < n ==> sorted[p] <= sorted[q] by {
  forall p, q | 0 <= p < i + 1 <= q < n
    ensures sorted[p] <= sorted[q]
  {

```

```

    if p < i {
      assert p != i && p != min;
      assert sorted[p] == before[p];
      if q == min {
        assert sorted[q] == before[i];
        assert before[p] <= before[i];
      } else {
        assert q != i;
        assert sorted[q] == before[q];
        assert before[p] <= before[q];
      }
    } else {
      assert p == i;
      assert sorted[p] == before[min];
      if q == min {
        assert sorted[q] == before[i];
        assert before[min] <= before[i];
      } else {
        assert q != i;
        assert sorted[q] == before[q];
        assert before[min] <= before[q];
      }
    }
  }
}

i := i + 1;
}

assert forall p, q :: 0 <= p < q < sorted.Length ==> sorted[p] <= sorted[q];
assert multiset(inputs[..]) == multiset(sorted[..]);
}

```

Final Program

```

method p_14_2_selection_sort_interval_array(inputs: array<int>, bound: nat) returns (
  ↪ sorted: array<int>)
  requires forall i :: 0 <= i < inputs.Length ==> 0 <= inputs[i] <= bound
  ensures forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]
  ensures multiset(inputs[..]) == multiset(sorted[..])
{
  var n := inputs.Length;
  sorted := new int[n];

  var k := 0;
  while k < n
    invariant 0 <= k <= n
    invariant n == inputs.Length
    invariant sorted.Length == n
    invariant forall t :: 0 <= t < k ==> sorted[t] == inputs[t]

```

```

{
  sorted[k] := inputs[k];
  k := k + 1;
}
assert forall t :: 0 <= t < n ==> sorted[t] == inputs[t];
assert sorted[..] == inputs[..];

var i := 0;
while i < n
  invariant 0 <= i <= n
  invariant n == inputs.Length
  invariant sorted.Length == n
  invariant multiset(sorted[..]) == multiset(inputs[..])
  invariant forall p, q :: 0 <= p < q < i ==> sorted[p] <= sorted[q]
  invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
{
  var min := i;
  var j := i + 1;
  while j < n
    invariant i + 1 <= j <= n
    invariant i <= min < j
    invariant sorted.Length == n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall p, q :: 0 <= p < q < i ==> sorted[p] <= sorted[q]
    invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] <= sorted[q]
    invariant forall t :: i <= t < j ==> sorted[min] <= sorted[t]
  {
    if sorted[j] < sorted[min] {
      var oldMin := min;
      assert sorted[j] < sorted[oldMin];
      min := j;
      assert forall t :: i <= t < j ==> sorted[min] <= sorted[t] by {
        forall t | i <= t < j
          ensures sorted[min] <= sorted[t]
        {
          assert sorted[min] < sorted[oldMin];
          assert sorted[oldMin] <= sorted[t];
        }
      }
      assert sorted[min] <= sorted[j];
    } else {
      assert sorted[min] <= sorted[j];
    }
  }
  assert forall t :: i <= t < j + 1 ==> sorted[min] <= sorted[t] by {
    forall t | i <= t < j + 1
      ensures sorted[min] <= sorted[t]
    {
      if t == j {
        assert sorted[min] <= sorted[j];
      } else {
        assert i <= t < j;
      }
    }
  }
}

```

```

    assert sorted[min] <= sorted[t];
  }
}
j := j + 1;
}

ghost var before := sorted[..];
assert |before| == n;
assert forall p, q :: 0 <= p < q < i ==> before[p] <= before[q];
assert forall p, q :: 0 <= p < i <= q < n ==> before[p] <= before[q];
assert forall t :: i <= t < n ==> before[min] <= before[t];

if min != i {
  assert i < min < n;
  sorted[i], sorted[min] := sorted[min], sorted[i];
  assert sorted[i] == before[min];
  assert sorted[min] == before[i];
  assert forall t :: 0 <= t < n && t != i && t != min ==> sorted[t] == before[t];
  assert sorted[..] == before[i := before[min]][min := before[i]];
  assert multiset(before[i := before[min]]) == multiset(before) - multiset{before[i]}
    ↪ + multiset{before[min]};
  assert multiset(before[i := before[min]][min := before[i]]) == multiset(before[i :=
    ↪ before[min]]) - multiset{before[min]} + multiset{before[i]};
  assert multiset(sorted[..]) == multiset(before);
} else {
  assert sorted[..] == before;
  assert sorted[i] == before[min];
  assert sorted[min] == before[i];
  assert forall t :: 0 <= t < n && t != i && t != min ==> sorted[t] == before[t];
  assert multiset(sorted[..]) == multiset(before);
}
assert multiset(sorted[..]) == multiset(inputs[..]);

assert forall p, q :: 0 <= p < q < i + 1 ==> sorted[p] <= sorted[q] by {
  forall p, q | 0 <= p < q < i + 1
    ensures sorted[p] <= sorted[q]
  {
    if q < i {
      assert p != i && p != min;
      assert q != i && q != min;
      assert sorted[p] == before[p];
      assert sorted[q] == before[q];
      assert before[p] <= before[q];
    } else {
      assert q == i;
      assert p < i;
      assert p != i && p != min;
      assert sorted[p] == before[p];
      assert sorted[q] == before[min];
      assert before[p] <= before[min];
    }
  }
}

```

```

    }
  }
}

assert forall p, q :: 0 <= p < i + 1 <= q < n ==> sorted[p] <= sorted[q] by {
  forall p, q | 0 <= p < i + 1 <= q < n
  ensures sorted[p] <= sorted[q]
{
  if p < i {
    assert p != i && p != min;
    assert sorted[p] == before[p];
    if q == min {
      assert sorted[q] == before[i];
      assert before[p] <= before[i];
    } else {
      assert q != i;
      assert sorted[q] == before[q];
      assert before[p] <= before[q];
    }
  } else {
    assert p == i;
    assert sorted[p] == before[min];
    if q == min {
      assert sorted[q] == before[i];
      assert before[min] <= before[i];
    } else {
      assert q != i;
      assert sorted[q] == before[q];
      assert before[min] <= before[q];
    }
  }
}
}

i := i + 1;
}

assert forall p, q :: 0 <= p < q < sorted.Length ==> sorted[p] <= sorted[q];
assert multiset(inputs[..]) == multiset(sorted[..]);
}

```

Total Token Usage

Input tokens: 358

Output tokens: 10107

Reasoning tokens: 8475

Sum of 'total tokens': 10465

Experiment Timings

Overall Experiment started at 1780669505281, ended at 1780669736279, lasting 230998ms (231.00 seconds)

Iteration #1 started at 1780669505282, ended at 1780669736278, lasting 230996ms (231.00 seconds)